

# Data Analysis: Recipes, Ingredients and their Average Rating

**Name(s):** Aishwarya Ramesh, Sai Poornasree Balamurugan

**Website Link:** [https://saib-poorna.github.io/Recipes\\_Reviews\\_Data\\_Analysis/](https://saib-poorna.github.io/Recipes_Reviews_Data_Analysis/)

## Imports

```
In [1]: import pandas as pd
import numpy as np
import os

import plotly.express as px
pd.options.plotting.backend = 'plotly'
```

## Cleaning and EDA

### Merging Dataframes

```
In [2]: # Getting data
recipes_raw = pd.read_csv("data/RAW_recipes.csv")
interactions_raw = pd.read_csv("data/RAW_interactions.csv")
```

```
In [3]: # Playing with recipes_raw
recipes_raw.shape
recipes_raw['id'].nunique()
# The number of unique recipes is 83,782
```

Out[3]: 83782

```
In [4]: # Playing with interactions_raw
interactions_raw.shape
# The number of interactions is much higher than the number of recipes
```

Out[4]: (731927, 5)

```
In [5]: # Merging data
# Ends up with df where each recipe ID can have multiple reviews, id can repeat
recipes_merged = recipes_raw.merge(interactions_raw, left_on = 'id', right_on = 'recipe_id', how = 'left')

# Replacing ratings of 0 with np.nan
recipes_merged['rating'] = recipes_merged['rating'].replace(0, np.nan)
# Why did we do this? It seems like 0 star reviews are allowed
```

```
In [6]: # Understanding recipes_merged
# 15036 recipes had 0 star ratings
recipes_merged['rating'].isna().sum()
```

Out[6]: 15036

```
In [7]: # DO NOT RUN CELL
# Drop recipe_id col as it is duplicate and rename id column
recipes_merged = recipes_merged.drop(columns = 'recipe_id')
recipes_merged = recipes_merged.rename(columns = {'id' : 'recipe_id'})

# Finding average rating per recipe as a Series
avg_ratings = recipes_merged.groupby('recipe_id')['rating'].mean()

avg_ratings = pd.DataFrame(avg_ratings).rename(columns = {'rating': 'avg_rating'})

# Adding series back to dataset
recipes = recipes_merged.merge(avg_ratings, left_on = 'recipe_id', right_index = True, how = 'left')
recipes.head()
```

Out[7]:

|   | name                               | recipe_id | minutes | contributor_id | submitted  | tags                                               | nutrition                                    | n_steps | steps                                             | description                                       | ingredients                                       | n_ingredients |
|---|------------------------------------|-----------|---------|----------------|------------|----------------------------------------------------|----------------------------------------------|---------|---------------------------------------------------|---------------------------------------------------|---------------------------------------------------|---------------|
| 0 | 1 brownies in the world best ever  | 333281    | 40      | 985201         | 2008-10-27 | ['60-minutes-or-less', 'time-to-make', 'course...] | [138.4, 10.0, 50.0, 3.0, 3.0, 19.0, 6.0]     | 10      | ['heat the oven to 350f and arrange the rack i... | these are the most; chocolatey, moist, rich, d... | ['bittersweet chocolate', 'unsalted butter', '... | 9             |
| 1 | 1 in canada chocolate chip cookies | 453467    | 45      | 1848091        | 2011-04-11 | ['60-minutes-or-less', 'time-to-make', 'cuisin...] | [595.1, 46.0, 211.0, 22.0, 13.0, 51.0, 26.0] | 12      | ['pre-heat oven the 350 degrees f', 'in a mixi... | this is the recipe that we use at my school ca... | ['white sugar', 'brown sugar', 'salt', 'margar... | 17            |
| 2 | 412 broccoli casserole             | 306168    | 40      | 50969          | 2008-05-30 | ['60-minutes-or-less', 'time-to-make', 'course...] | [194.8, 20.0, 6.0, 32.0, 22.0, 36.0, 3.0]    | 6       | ['preheat oven to 350 degrees', 'spray a 2 qua... | since there are already 411 recipes for brocco... | ['frozen broccoli cuts', 'cream of chicken sou... | 9             |
| 3 | 412 broccoli casserole             | 306168    | 40      | 50969          | 2008-05-30 | ['60-minutes-or-less', 'time-to-make', 'course...] | [194.8, 20.0, 6.0, 32.0, 22.0, 36.0, 3.0]    | 6       | ['preheat oven to 350 degrees', 'spray a 2 qua... | since there are already 411 recipes for brocco... | ['frozen broccoli cuts', 'cream of chicken sou... | 9             |
| 4 | 412 broccoli casserole             | 306168    | 40      | 50969          | 2008-05-30 | ['60-minutes-or-less', 'time-to-make', 'course...] | [194.8, 20.0, 6.0, 32.0, 22.0, 36.0, 3.0]    | 6       | ['preheat oven to 350 degrees', 'spray a 2 qua... | since there are already 411 recipes for brocco... | ['frozen broccoli cuts', 'cream of chicken sou... | 9             |

Question: Do recipes with more or less ingredients have higher average ratings?

In [8]:

```
# Columns we need: n_ingredients and rating
# Checking data type

recipes['n_ingredients'].dtypes # integer
recipes['rating'].dtypes # float
```

Out[8]: dtype('float64')

Univariate Analysis of n\_ingredients

In [9]:

```
### Getting unique values for the number of ingredients
num_ingredients = recipes.groupby('recipe_id').mean()['n_ingredients']
```

In [10]:

```
# Getting the distribution of the number of ingredients
num_ingredients.describe()
```

Out[10]:

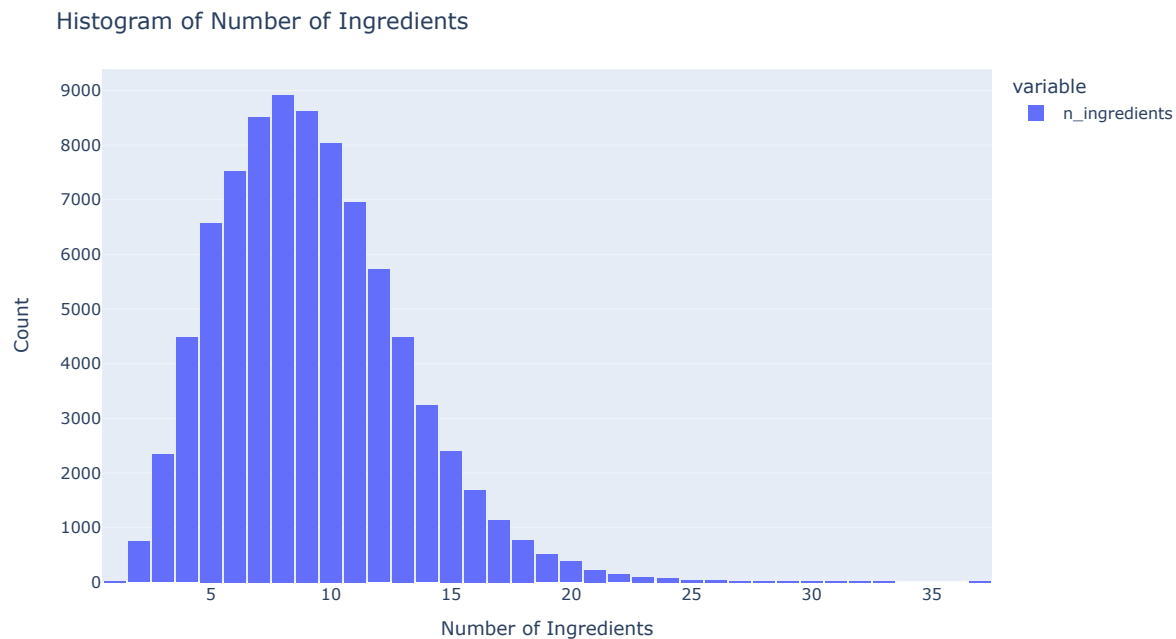
|       |              |
|-------|--------------|
| count | 83782.000000 |
| mean  | 9.214020     |
| std   | 3.830465     |
| min   | 1.000000     |
| 25%   | 6.000000     |
| 50%   | 9.000000     |
| 75%   | 12.000000    |
| max   | 37.000000    |

Name: n\_ingredients, dtype: float64

In [11]:

```
# Plotting distribution of n_ingredients
fig = px.histogram(num_ingredients)
fig.update_layout(
    title = 'Histogram of Number of Ingredients',
    xaxis_title = 'Number of Ingredients', yaxis_title = 'Count', bargap = 0.1)

fig.show()
```



The distribution of `n_ingredients` has a mean of 9.21, and is slightly right skewed but appears to be approximately normally distributed.

#### Univariate Analysis of `avg_rating`

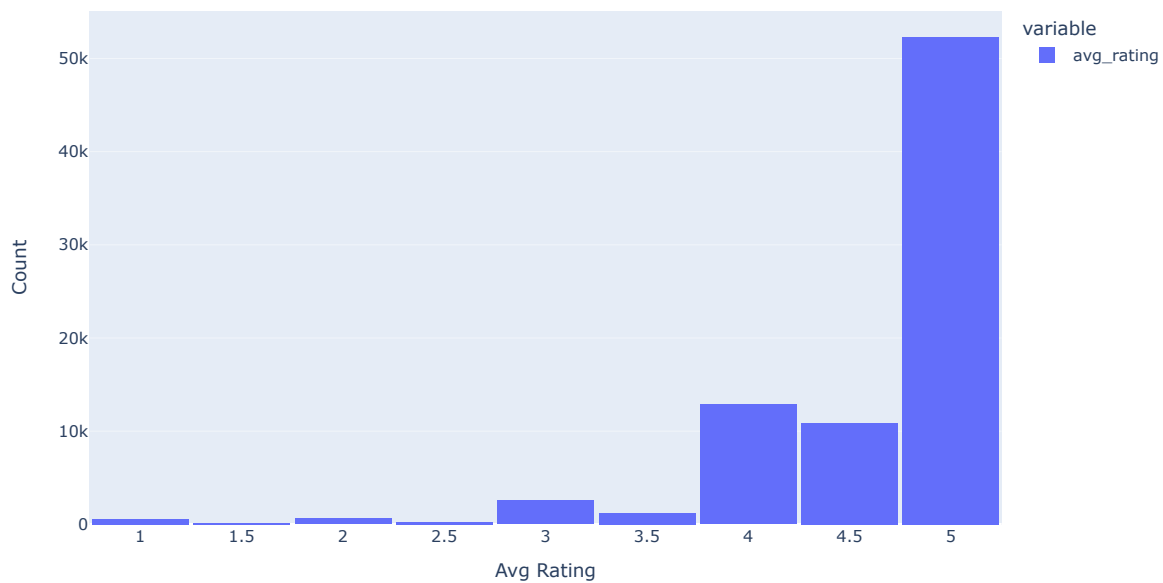
```
In [12]: # Getting unique values for avg_rating
avg_ratings = recipes.groupby('recipe_id')['avg_rating'].mean()
```

```
In [13]: # Getting the distribution of avg_ratings
avg_ratings.describe()
```

```
Out[13]: count    81173.000000
mean         4.625363
std          0.640763
min          1.000000
25%          4.500000
50%          5.000000
75%          5.000000
max          5.000000
Name: avg_rating, dtype: float64
```

```
In [14]: # Plotting avg_ratings histogram
fig = px.histogram(avg_ratings, nbins = 10)
fig.update_layout(
    title = 'Histogram of Avg Ratings',
    xaxis_title = 'Avg Rating', yaxis_title = 'Count', bargap = 0.05)
fig.show()
```

Histogram of Avg Ratings

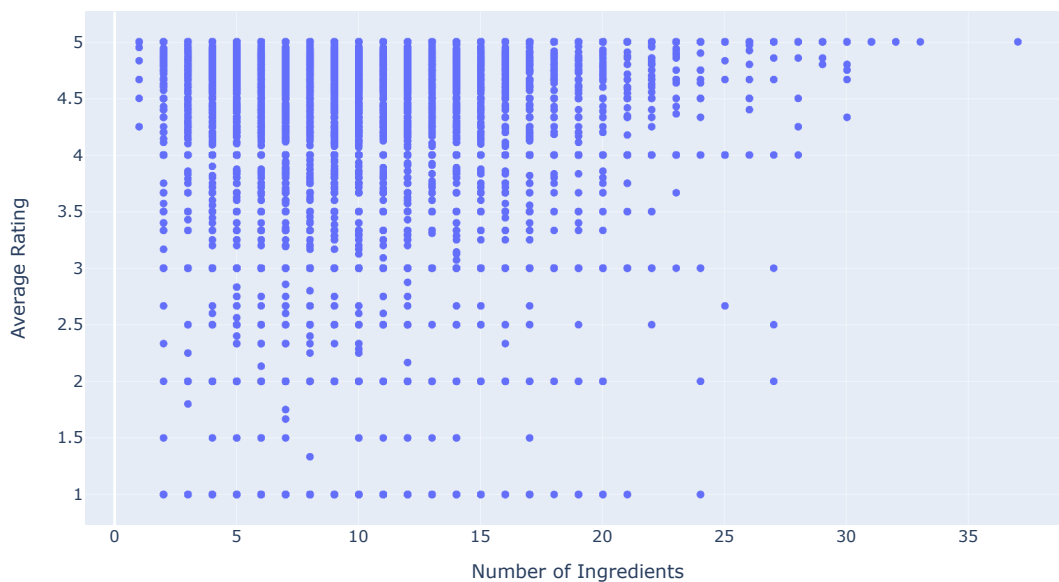


From the histogram of average ratings we observe that most dishes are highly rated, with the peak of the histogram being at a rating of 5.

#### Bivariate Analysis of n\_ingredients and avg\_rating

```
In [15]: # Creating a scatterplot of n_ingredients and avg_rating
scatter_fig = px.scatter(x=num_ingredients, y=avg_ratings)
scatter_fig.update_layout(
    title = 'Scatterplot of Number of Ingredients vs Average Rating',
    xaxis_title = 'Number of Ingredients', yaxis_title = 'Average Rating')
scatter_fig.show()
```

Scatterplot of Number of Ingredients vs Average Rating



From the scatterplot above, we can observe that recipes with less ingredients seem to have more high ratings. However, this might also be because there are more data between 5-20 ingredients than beyond that point. Hypothesis tests will need to be done to conclude whether recipes with lower or higher ingredients get higher ratings.

#### Interesting Aggregates -- Pivot Table

We can add a few columns to recipes that categorize the data by rating or num ingredients and use them for the pivot table

```
In [16]: # Categorizing n_ingredients
ingredient_categories = ['low' if num <= 9 else 'high' for num in recipes['n_ingredients']]
recipes = recipes.assign(ing_size_label = ingredient_categories)
```

```
In [17]: # Categorizing the ratings

def rating_categorizer(rating):
    if rating <= 1:
        return '1 star or less'
    elif rating <= 2:
        return '1 to 2 star rating'
    elif rating <= 3:
        return '2 to 3 star rating'
    elif rating <= 4:
        return '3 to 4 star rating'
    else:
        return '4 to 5 star rating'

recipes['rating_categories'] = recipes['avg_rating'].apply(rating_categorizer)
```

```
In [18]: # Creating a pivot table that tells us number of reviews within each category of ingredients
recipes.pivot_table(
    index = 'ing_size_label',
    columns = 'rating_categories',
    values = 'avg_rating',
    aggfunc = 'count'
)
```

```
Out[18]: rating_categories  1 star or less  1 to 2 star rating  2 to 3 star rating  3 to 4 star rating  4 to 5 star rating
ing_size_label
high          292          327          1641          9975          82594
low           405          462          2328          13432          120196
```

## Assessment of Missingness

```
In [19]: # NMAR
# Missingness of data depend on data themselves
# Why are there so much more 5 star ratings than lower ratings -- ppl more likely to review if they liked it
```

Finding dependency of columns

```
In [20]: # Evaluating missingness within each column of the dataset
recipes.isna().sum()
```

```
Out[20]: name          1
recipe_id         0
minutes           0
contributor_id    0
submitted         0
tags              0
nutrition         0
n_steps           0
steps             0
description       114
ingredients       0
n_ingredients     0
user_id           1
date              1
rating           15036
review            58
avg_rating        2777
ing_size_label    0
rating_categories  0
dtype: int64
```

It seems like the column with the most missing data is rating. We can check its dependency on other columns using a permutation test. A good test statistic to use would be the absolute difference of means.

```
In [21]: # Function to calculate absolute difference of means
def abs_mean_diff(ser1, ser2):
    return abs(ser1.mean() - ser2.mean())
```

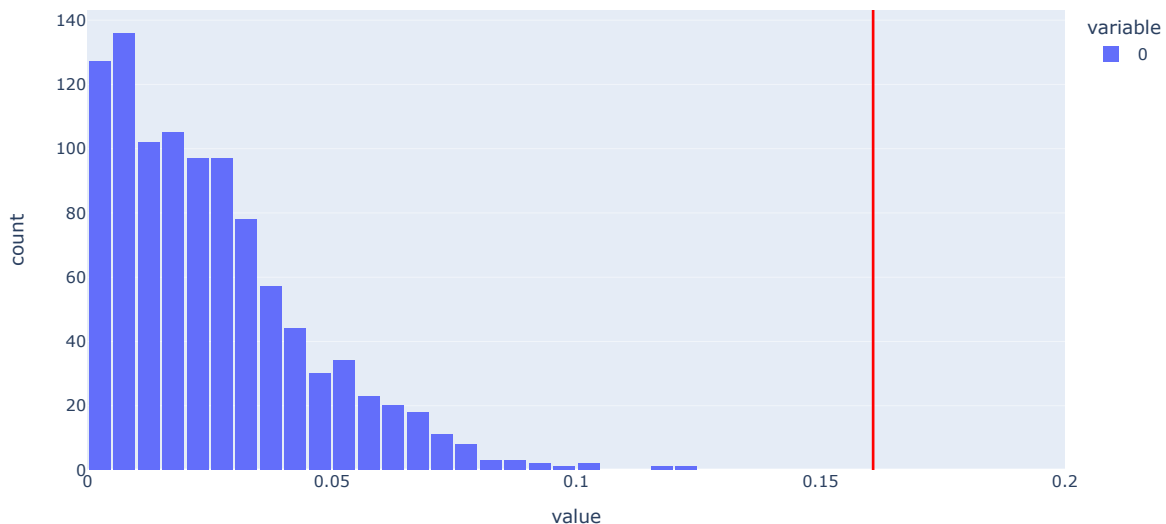
```
In [22]: # Function for permutation test
def perm_test_missing(df, n_repetitions, missing_col, permuting_col):
    shuffled = df.copy()
    shuffled['missing'] = shuffled[missing_col].isna()
    # Calculating observed stat
    missing_data = shuffled[shuffled['missing'] == True][permuting_col]
    non_missing_data = shuffled[shuffled['missing'] == False][permuting_col]
    observed_abs_mean_diff = abs_mean_diff(missing_data, non_missing_data)
    stats = []
    # Getting test stats
    for i in range(n_repetitions):
        shuffled['missing'] = np.random.permutation(shuffled['missing'])
        missing_data = shuffled[shuffled['missing'] == True][permuting_col]
        non_missing_data = shuffled[shuffled['missing'] == False][permuting_col]
        stat = abs_mean_diff(missing_data, non_missing_data)
        stats.append(stat)
    # Calculating p-value
    p_val = np.mean(np.array(stats) >= observed_abs_mean_diff)
    return [p_val, np.array(stats), observed_abs_mean_diff]
```

```
In [23]: # Running the permutation test on rating and n_ingredients
n_ingredients_perm = perm_test_missing(recipes, 1000, 'rating', 'n_ingredients')
print('p-value: ', n_ingredients_perm[0])
print('Observed test stat : ', n_ingredients_perm[2])
```

```
p-value: 0.0
Observed test stat : 0.1607379066254797
```

```
In [24]: # Plotting the results of the permutation test
perm_dep_fig = px.histogram(n_ingredients_perm[1], title='Distribution of Test Statistics (Absolute Difference of Means)')
perm_dep_fig.add_vline(x=n_ingredients_perm[2], line_color='red', line_width=2, opacity=1)
perm_dep_fig.update_layout(xaxis_range=[0, 0.2], bargap=0.1)
```

Distribution of Test Statistics (Absolute Difference of Means)



From the results of the permutation test, we reject the null hypothesis that there is no relationship between the missingness of `rating` and `n_ingredients` column. We can conclude that `rating` seems to be MAR with respect to `n_ingredients`. The red line above indicates the observed test statistic of 0.16, which is clearly far beyond any of the simulated stats.

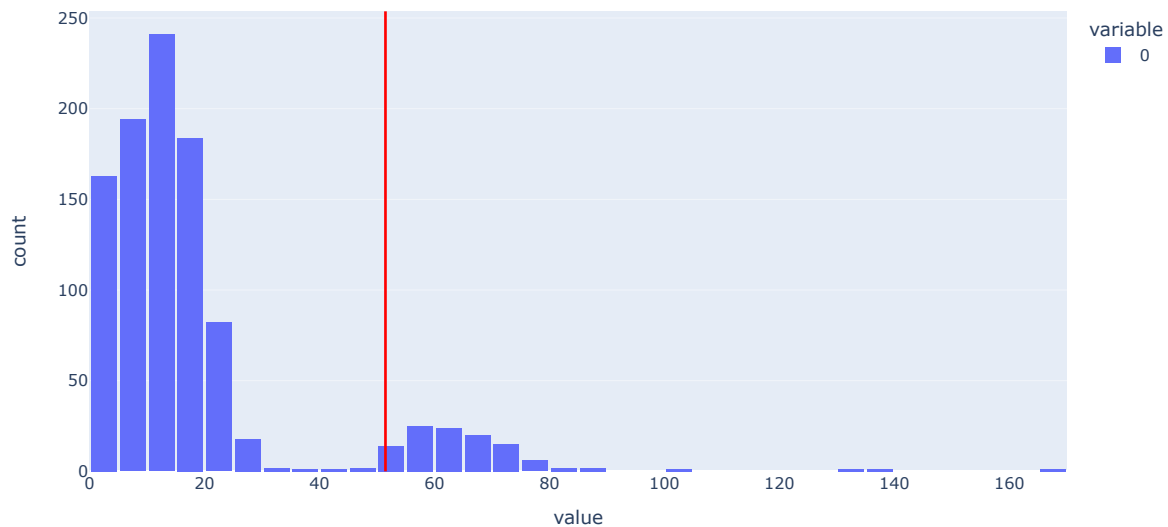
```
In [25]: # Checking if missingness depends on cooking time in minutes column
mins_perm_test = perm_test_missing(recipes, 1000, 'rating', 'minutes')
mins_perm_test
print('p-value: ', mins_perm_test[0])
print('Observed test stat : ', mins_perm_test[2])
```

```
p-value: 0.109
Observed test stat : 51.45237039852127
```

```
In [26]: # Plotting the results of the permutation test
perm_indep_fig = px.histogram(mins_perm_test[1], title='Distribution of Test Statistics (Absolute Difference of Means)')
perm_indep_fig.add_vline(x=mins_perm_test[2], line_color='red', line_width=2, opacity=1)
```

```
perm_indep_fig.update_layout(bargap = 0.1)
```

Distribution of Test Statistics (Absolute Difference of Means)



From the permutation test and figure above, we fail to reject the null hypothesis that the missingness of `rating` does not depend on `minutes`. That is, the missingness of `rating` does not depend on the values of the `minutes` column. We got a p-value of around 0.13, which is greater than 0.05. Clearly, our observed test stat (red line) falls in the range of the simulated test stats.

## Hypothesis Testing

**Null Hypothesis:** There is no significant difference in average rating between recipes that have a lower number of ingredients vs a higher number of ingredients.

**Alternate:** There is a significant difference in average rating between recipes that have a lower number of ingredients vs a higher number of ingredients.

```
In [27]: # First, getting the columns we need for the permutation test
hyp_df = recipes[['recipe_id', 'avg_rating', 'ing_size_label']]
hyp_df = hyp_df.drop_duplicates()
hyp_df
```

```
Out[27]:
```

|        | recipe_id | avg_rating | ing_size_label |
|--------|-----------|------------|----------------|
| 0      | 333281    | 4.0        | low            |
| 1      | 453467    | 5.0        | high           |
| 2      | 306168    | 5.0        | low            |
| 6      | 286009    | 5.0        | low            |
| 7      | 475785    | 5.0        | high           |
| ...    | ...       | ...        | ...            |
| 234413 | 486161    | 5.0        | high           |
| 234419 | 493372    | 5.0        | high           |
| 234420 | 308080    | 5.0        | low            |
| 234425 | 298512    | 1.0        | high           |
| 234426 | 298509    | 3.0        | low            |

83782 rows × 3 columns

```
In [28]: # Hypothesis test
low = hyp_df[hyp_df['ing_size_label'] == 'low']['avg_rating']
high = hyp_df[hyp_df['ing_size_label'] == 'high']['avg_rating']
# Calculating observed test stat
observed_test_stat = abs_mean_diff(low, high)
```

```

hyp_df['ing_size_label_shuffled'] = hyp_df['ing_size_label']
# Getting simulated stats
stats = []
n_repetitions = 10_000
for i in range(n_repetitions):
    hyp_df['ing_size_label_shuffled'] = np.random.permutation(hyp_df['ing_size_label_shuffled'])
    low_shuffled = hyp_df[hyp_df['ing_size_label_shuffled'] == 'low']['avg_rating']
    high_shuffled = hyp_df[hyp_df['ing_size_label_shuffled'] == 'high']['avg_rating']
    stat = abs_mean_diff(low_shuffled, high_shuffled)
    stats.append(stat)
# Calculating p-value
p_val = (np.array(stats) >= observed_test_stat).mean()

```

```
In [29]: print('Hypothesis test p-value: ', p_val)
```

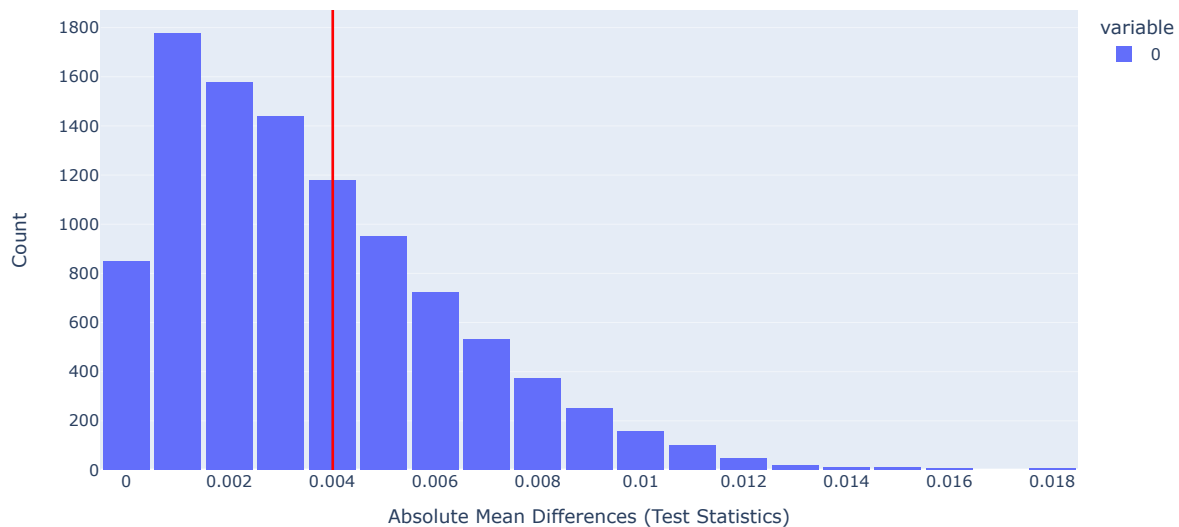
Hypothesis test p-value: 0.373

```

In [30]: # Plotting the results of the hypothesis test
hyp_fig = px.histogram(stats, title='Distribution of Simulated Test Statistics (Absolute Difference of Mean)', nbins = 30)
hyp_fig.add_vline(x=observed_test_stat, line_color = 'red', line_width = 2, opacity = 1)
hyp_fig.update_layout(xaxis_title = 'Absolute Mean Differences (Test Statistics)', yaxis_title = 'Count', bargap = 0.1)

```

Distribution of Simulated Test Statistics (Absolute Difference of Mean)



From the p-value and the graph above, we fail to reject the null hypothesis that there is no significant difference in average rating between recipes with less ingredients or more ingredients. This is because the p-value was 0.3797, which is much higher than the threshold of 0.05. From the graph, we can see that our observed test statistic (red line) is within the bounds of simulated test statistics as well.

```
In [ ]:
```